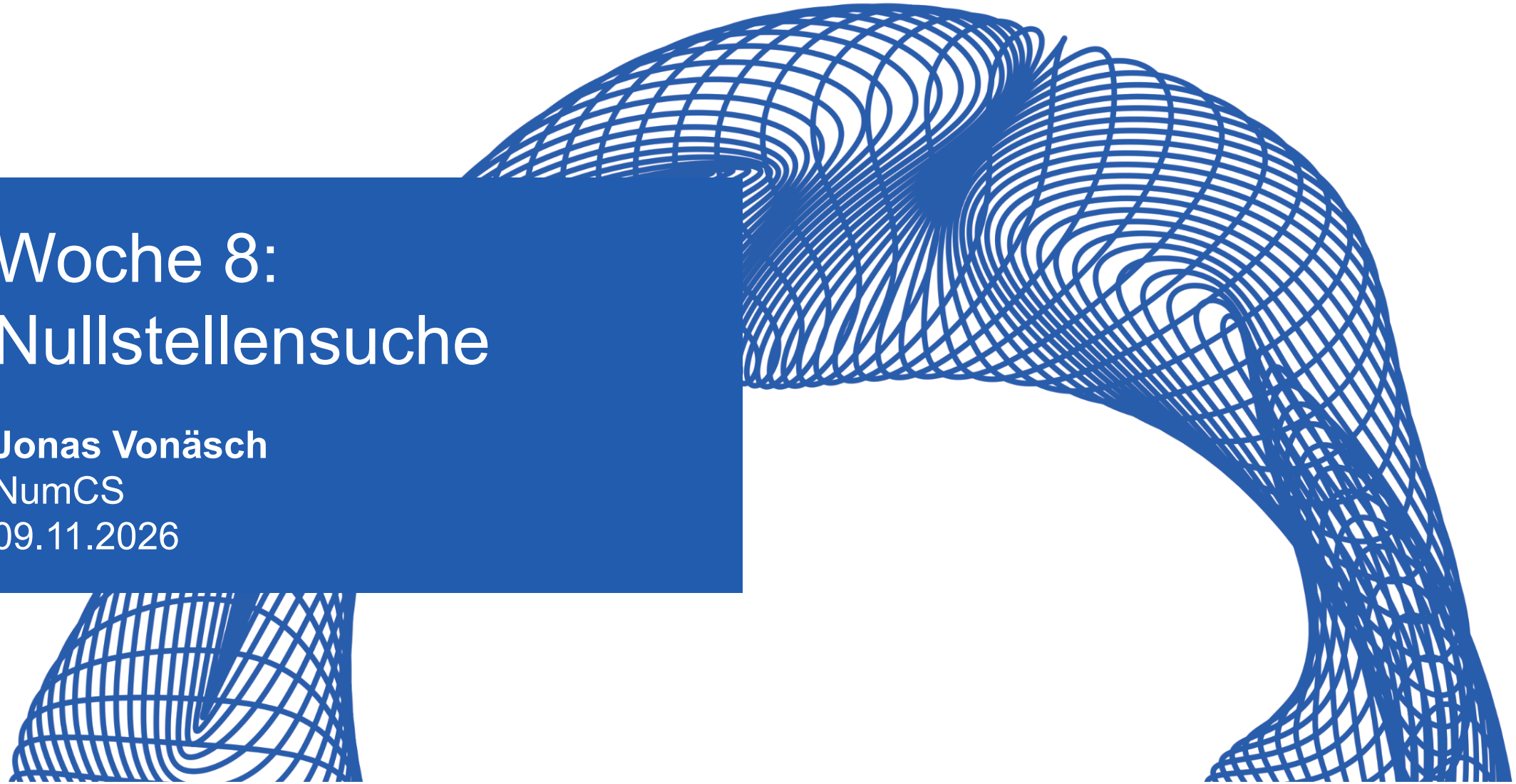
ETH zürich



Woche 8: Nullstellensuche

Jonas Vonäsch
NumCS
09.11.2026

Agenda

1. Nullstellensuche
    1. Iterative Verfahren
    2. Fixpunktiteration
        1. Intervallhalbierungsverfahren
        2. Newton-Verfahren
        3. Sekanten-Verfahren
        4. Newton d-dimensional
        5. Quasi-Newton

Nullstellensuche

Es steht im Titel…

Gegeben sei Funktion $\mathbf{F}$ definiert auf einem Teil von $\mathbb{R}^d$:

$$\mathbf{F} : \Omega \subset \mathbb{R}^d \to \mathbb{R}^d, \quad d \in \mathbb{N}$$

Wir suchen $\mathbf{x}^* \in \Omega$ sodas:

$$\mathbf{F}(\mathbf{x}^*) = 0$$

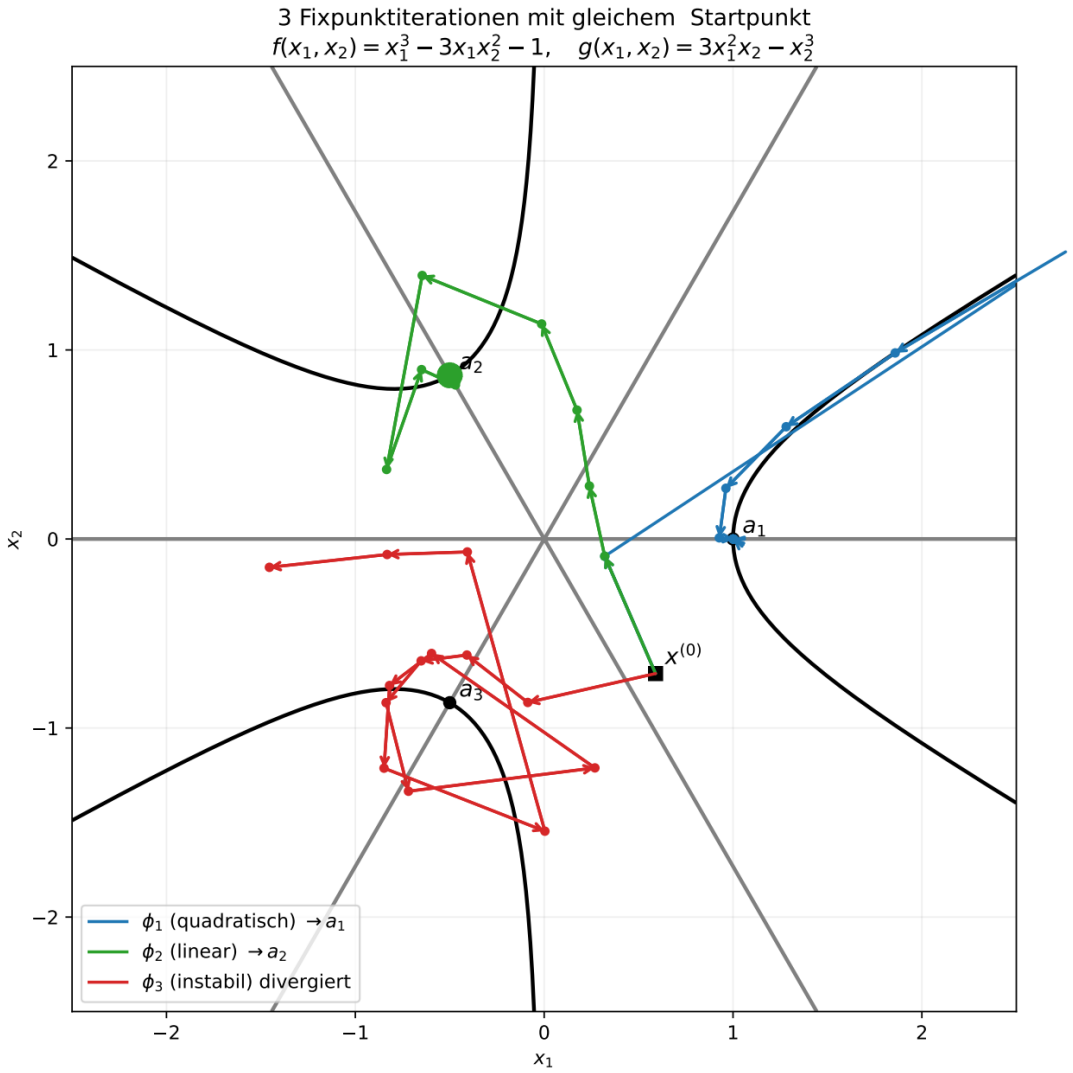
Iterative Nullstellensuche

Iterative Verfahren: Definition

Ein Algorithmus $\phi_{\mathbf{F}}$, der die Folge $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots$ generiert, um $\mathbf{x}^*$ anzunähern.

Die Definition von $\mathbf{x}^{(k)}$ erfolgt **rekursiv**:

$$\mathbf{x}^{(k)} := \phi\left(\mathbf{x}^{(k-1)}\right)$$



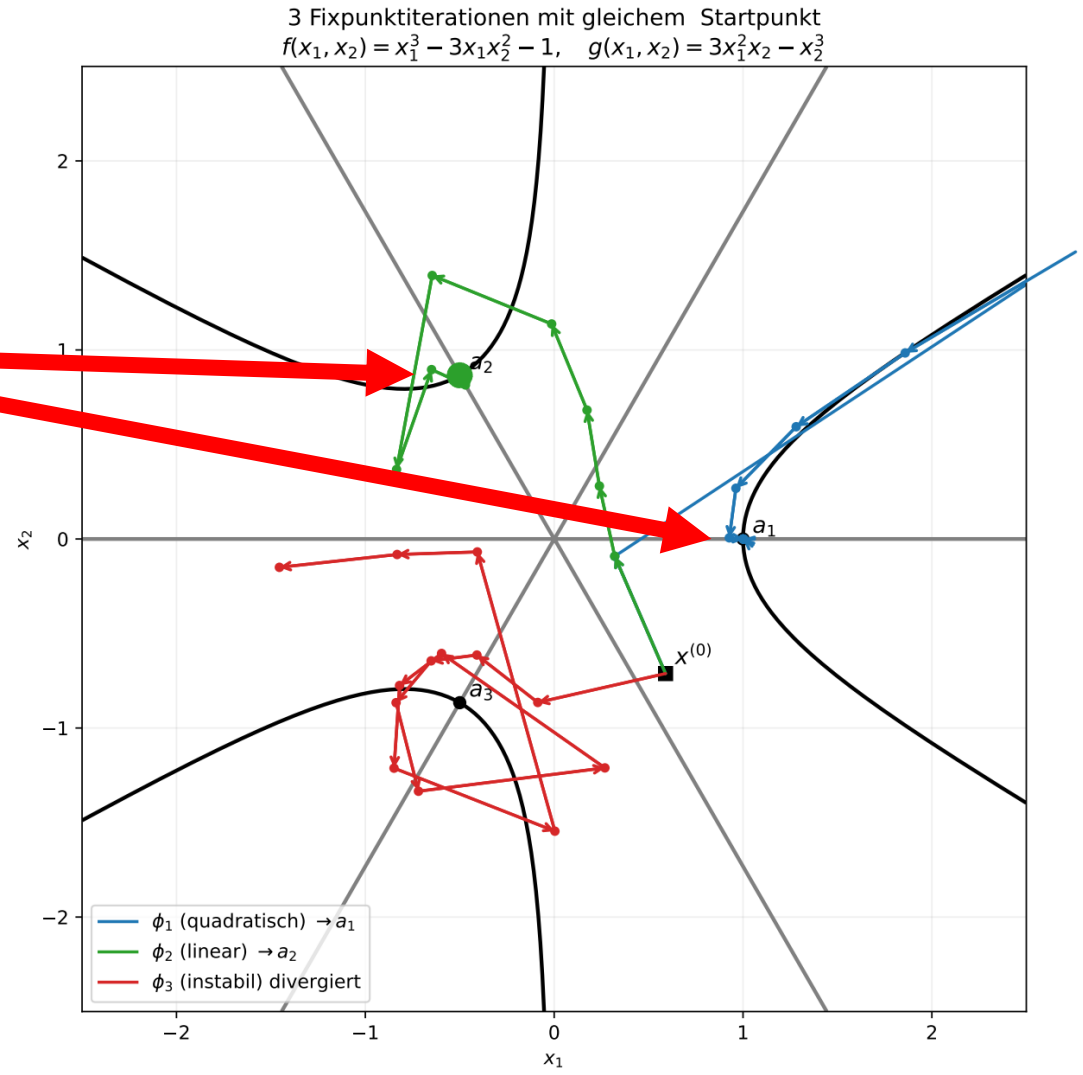
Iterative Verfahren: Konvergenz

Ein iteratives Verfahren $\phi_{\mathbf{F}}$ zur Nullstellensuche auf $\mathbf{F}$ konvergiert, falls:

$$\mathbf{x}^{(k)} \to \mathbf{x}^* \in \Omega, \quad \mathbf{F}(\mathbf{x}^*) = 0$$

- Dies kann vom Startwert $\mathbf{x}^{(0)}$ abhängen, den wir (wenn möglich) *Nahe einer Nullstelle* wählen sollten

- Je nach $\mathbf{x}^{(0)}$ findet $\phi_{\mathbf{F}}$ unterschiedliche Lösungen!



Wie schnell konvergiert ϕ?

Wann soll ich aufhören?

Ein Wenig Fachjargon…

Definition: Norm

Sei $X \in \mathbb{K}$ mit $\mathbb{K} = \mathbb{R}$ oder $\mathbb{K} = \mathbb{C}$.

Eine Abbildung $\|\cdot\| : X \mapsto \mathbb{R}_0^+$ heisst *norm* (z.B. Euklidisch Norm), wenn:

i. $\forall \mathbf{x} \in X : \mathbf{x} \neq 0$ genau dann, wenn $\|\mathbf{x}\| > 0$

ii. $\|\lambda \mathbf{x}\| = |\lambda| \|\mathbf{x}\|$ für alle $\mathbf{x} \in X$ und alle $\lambda \in \mathbb{K}$

iii. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ für alle $\mathbf{x}, \mathbf{y} \in X$.

Name	:	Definition	`numpy.linalg`
Euklidische Norm	:	$\|\mathbf{x}\|_2 := \sqrt{\|x_1\|^2 + \cdots + \|x_d\|^2}$	`norm(x)`
1-Norm	:	$\|\mathbf{x}\|_1 := \|x_1\| + \cdots + \|x_d\|$	`norm(x,1)`
∞-Norm, Max-Norm	:	$\|\mathbf{x}\|_\infty := \max\{\|x_1\|, \ldots, \|x_d\|\}$	`norm(x,inf)`

Alle Normen im endlich-dimensionalen Raum sind *Äquivalent*!

$$\exists \underline{C}, \overline{C} \text{ sodass gilt:} \quad \forall v \in X \cdot \underline{C}\|v\|_1 \leq \|v\|_2 \leq \overline{C}\|v\|_1$$

Wir können beliebige Normen verwenden, um dieselben Konvergenzaussagen zu treffen.

Iterative Nullstellensuche: Konvergenz

- Definition **Konvergenzrate**: Wenn $\mathbf{x}^{(k)}$ erfüllt, dass

$$\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\right\| \leq L\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\| \quad , \forall k \geq k_0$$

  Für ein $L < 1$, nennen wir L die *Konvergenzrate*.

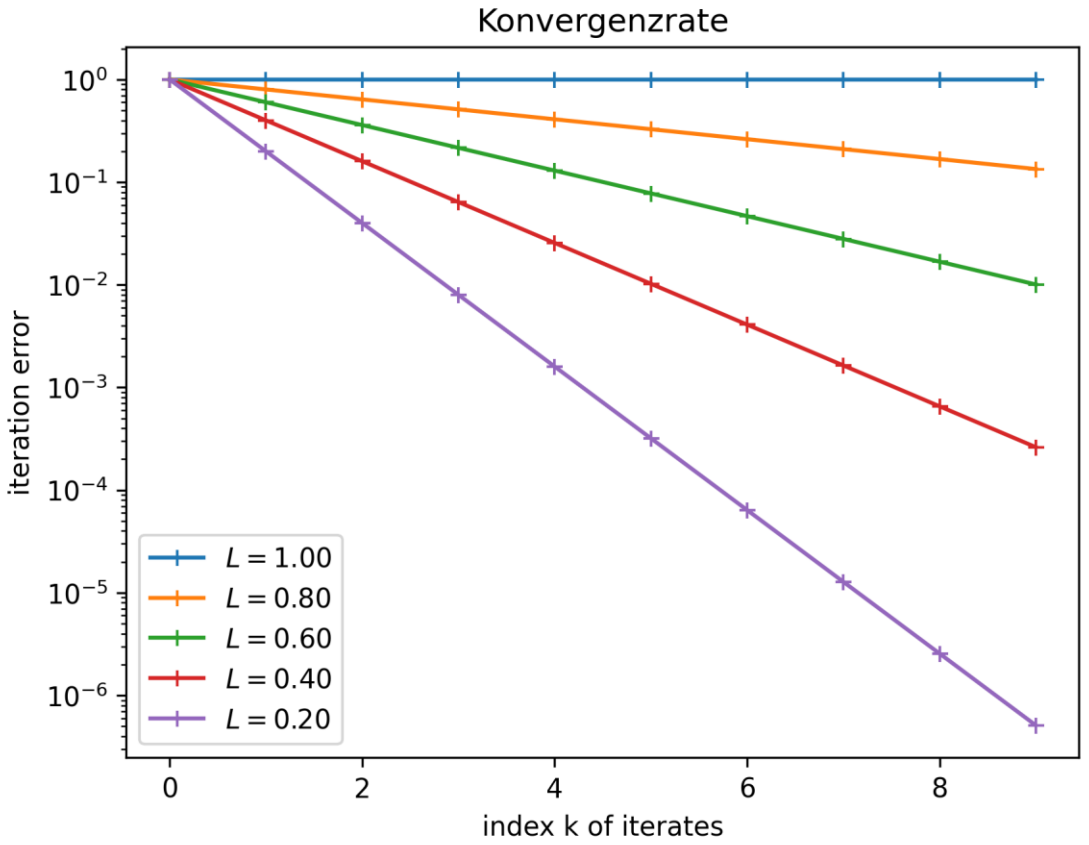
- Definition **Konvergenzordnung**: Wenn $\mathbf{x}^{(k)}$ erfüllt, dass

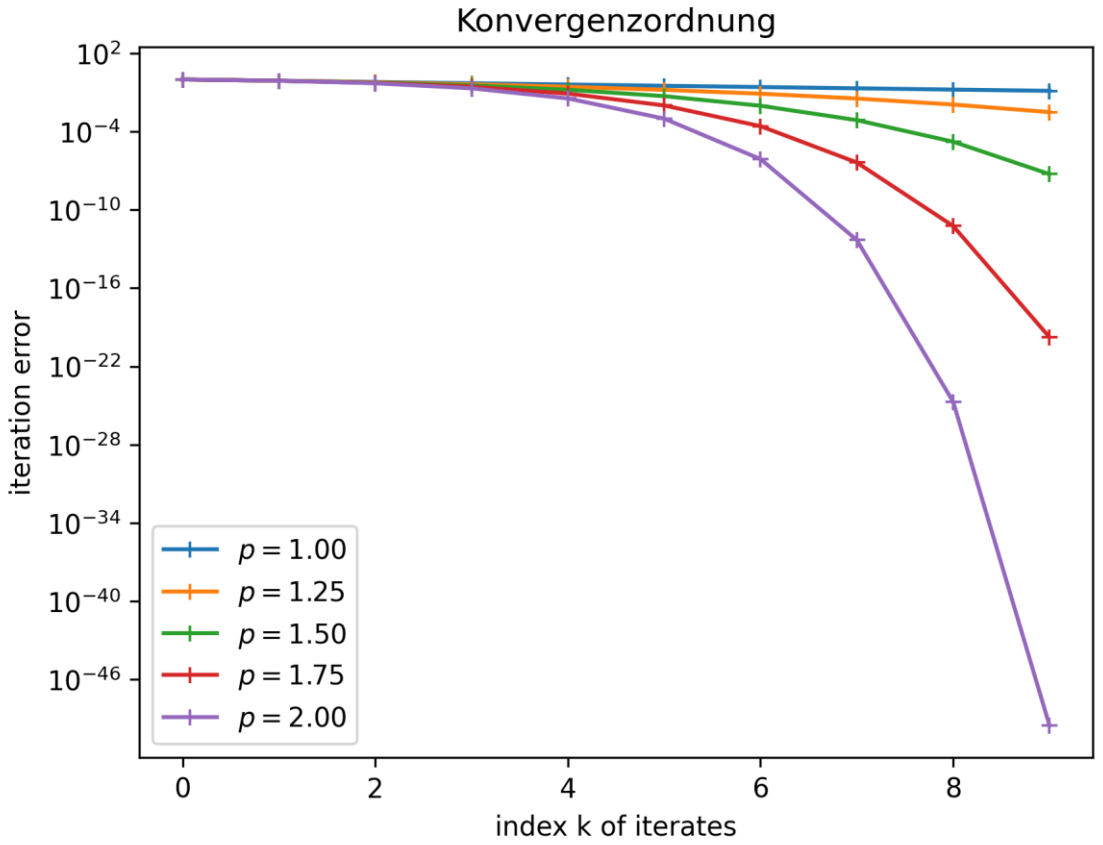
$$\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\right\| \leq C\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\|^p \quad , \forall k \in \mathbb{N}$$

  Für ein $C > 0$, nennen wir p die *Konvergenzordnung*.

Spezialfall: Wenn $p = 1$, dann $C < 1$ (Konvergenzordnung 1 $\Longrightarrow$ Konvergenzrate C)

Konvergenzrate vs. Konvergenzordnung



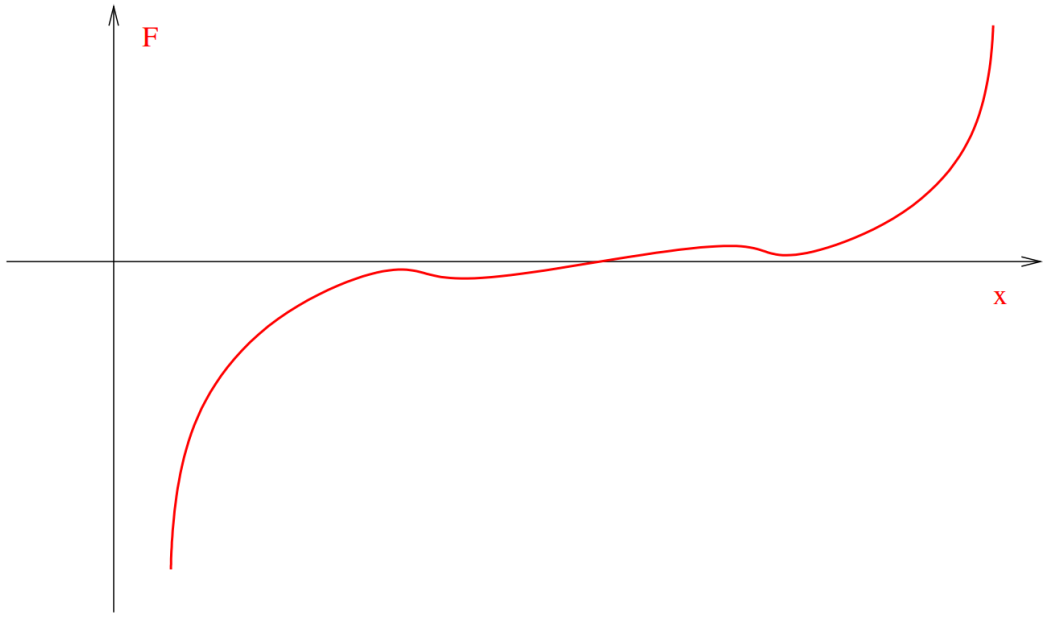


Iterative Nullstellensuche: Wann hören wir auf?

- A *priori* Kriterium: Nach genau k_0 Schritten
- A *posteriori* Kriterium: $\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\| < \tau$ (nicht praktikabel, da $\mathbf{x}^*$ unbekannt)*
- Durchführen, bis $\mathbf{x}^{(k+1)} \approx \mathbf{x}^{(k)}$
- Residuum: $\left\|\mathbf{F}\left(\mathbf{x}^{(k)}\right)\right\| < \tau$ sehr intuitiv, aber Problem:
- Für lineare Konvergen:

$$\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\right\| \leq \frac{L}{1-L} \|\mathbf{x}^($$

*damit funktioniert a pc



Fixpunktiteration

Fixpunktiteration

Wir schreiben $\boldsymbol{F(x) = 0}$ *um als* $\boldsymbol{\phi(x) = x}$

Intuitiv: Wenn wir stehenbleiben, sind wir am Ziel angekommen!

Definition (1-Punkt-Verfahren): Benötigt nur den jeweils letzten Punkt

$$\mathbf{x}^{(k+1)} = \phi\left(\mathbf{x}^{(k)}\right)$$

Definition (Konsistenz): Fixpunktiteration eisst konsistent, falls

$$\mathbf{F}(\mathbf{x}) = 0 \Leftrightarrow \phi(\mathbf{x}) = \mathbf{x}$$

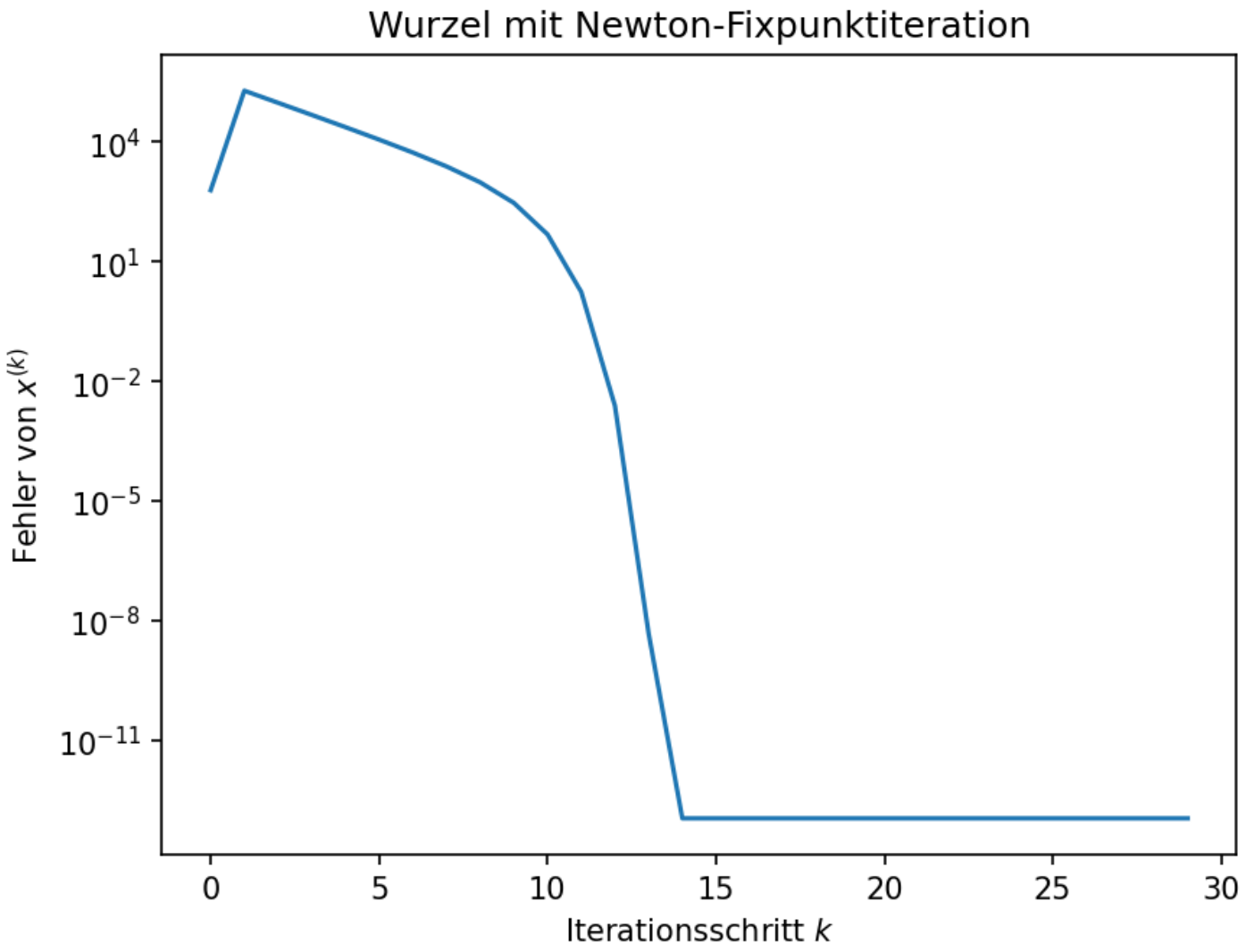
Fixpunktiteration: Beispiel

Newton's Verfahren zum Berechnen einer Quadratwurzel:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

Ist eine Fixpunktiteration zur Nullstellensuche für $(\mathbf{x}^*)^2 - a = 0$

Fixpunktiteration: Beispiel

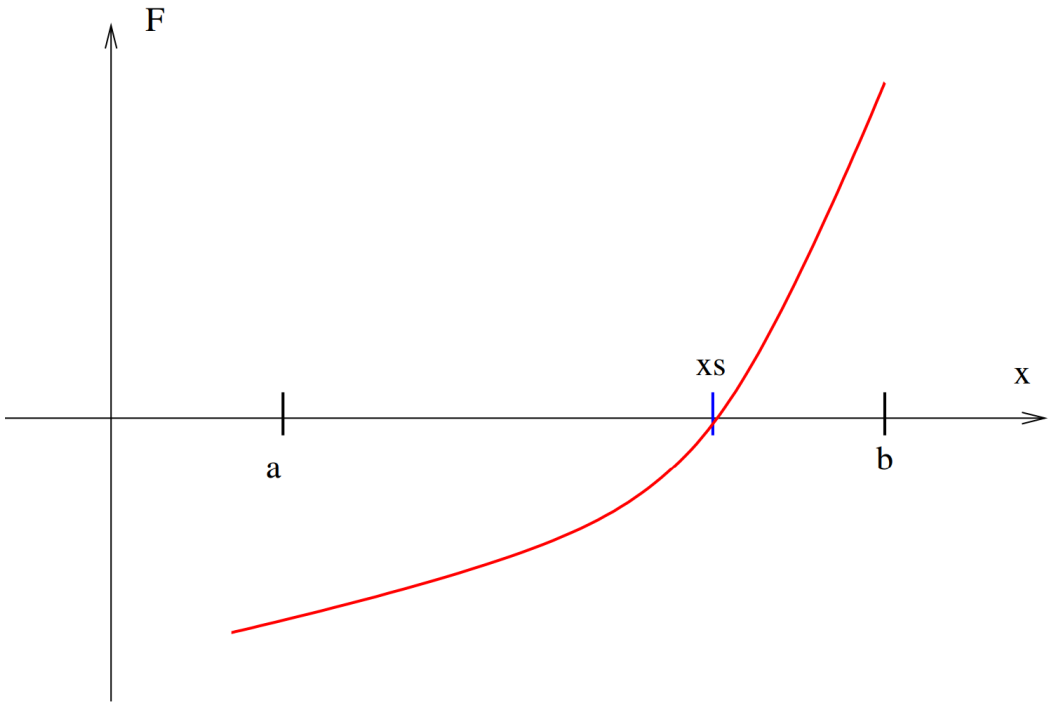


Intervallhalbierungsverfahren (Bisektionsverfahren)

Recap Zwischenwertsatz

Für F stetig auf $[a, b]$ und $F(a)F(b) < 0$ (unterschiedliche Vorzeichen an a und b):

$$\exists x^* \in [a, b] \quad \text{für das gilt} \quad F(x^*) = 0$$



Intervallhalbierungsverfahren: Vorgehen

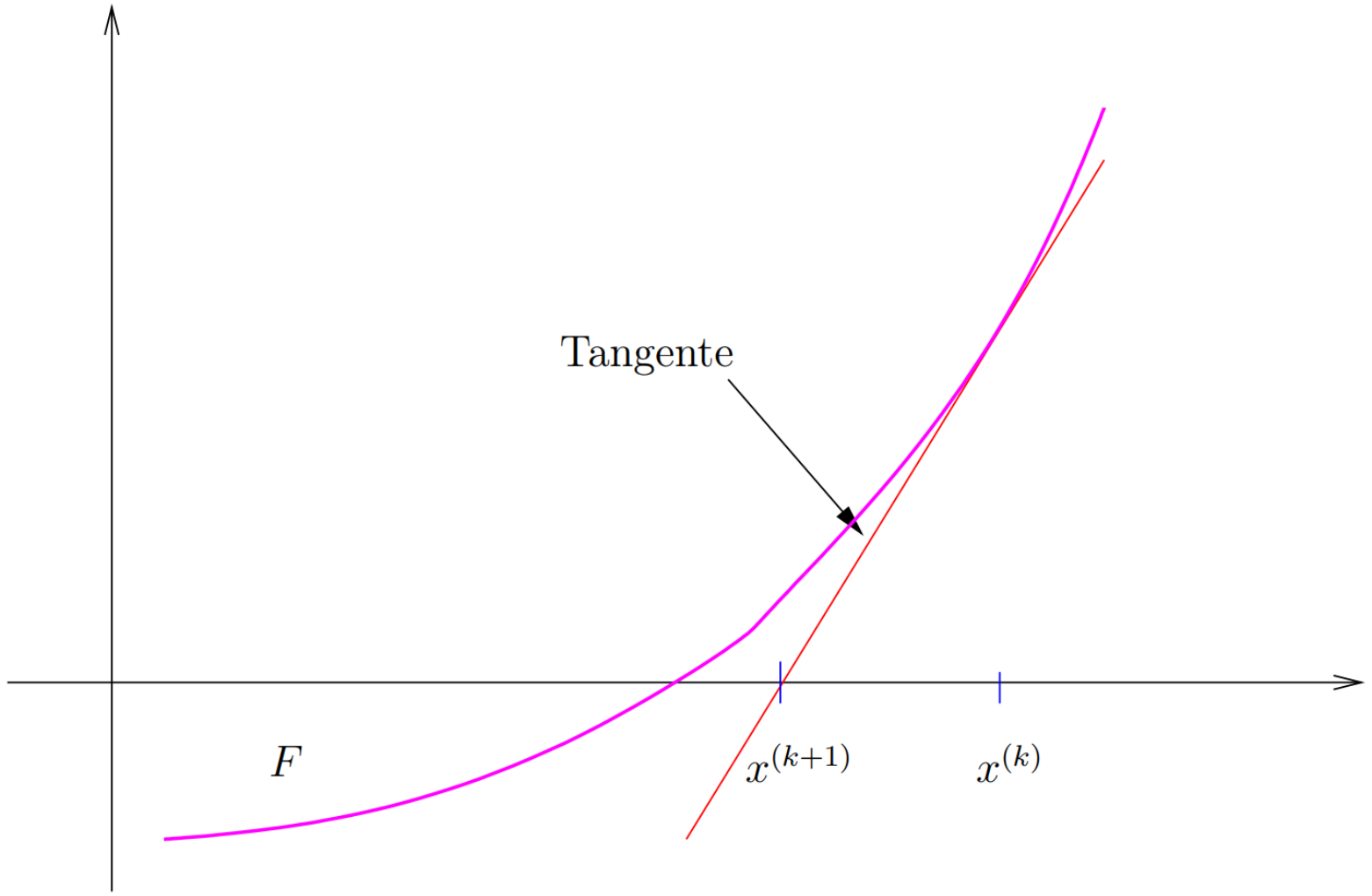
1. Suche $[a, b]$ so, dass $F(a)F(b) < 0$ (unterschiedliche Vorzeichen)
2. Wiederhole bis Abbruch:
   1. Messe an Stelle $c = \frac{a+b}{2}$
   2. Wenn $F(c) = 0$ oder Kriterium erfüllt, breche ab
   3. Wenn $F(a)F(c) < 0$, setze $b = c$, sonst setze $a = c$

Das Verfahren halbiert in jeder Iteration das Intervall, auf dem wir suchen, und die Nullstelle liegt jeweils im Intervall!

Nachteil: Wir brauchen $[a, b]$ mit $F(a)F(b) < 0$ – Schwer zu finden!

Newton-Verfahren 1D

Newton-Verfahren: Intuition



Newton-Verfahren: Vorgehen

Die Formel für die Tangente ist: $\tilde{F}(x) = F\left(x^{(k)}\right) + F'\left(x^{(k)}\right)\left(x - x^{(k)}\right)$

Wir setzen diese gleich 0 und lösen nach x:

$$F\left(x^{(k)}\right) + F'\left(x^{(k)}\right)\left(x - x^{(k)}\right) = 0 \quad \Rightarrow \quad -F\left(x^{(k)}\right) = F'\left(x^{(k)}\right)\left(x - x^{(k)}\right)$$

$$\Rightarrow \quad -\frac{F\left(x^{(k)}\right)}{F'\left(x^{(k)}\right)} = x - x^{(k)}$$

$$\Rightarrow \quad x^{(k)} - \frac{F\left(x^{(k)}\right)}{F'\left(x^{(k)}\right)} = x$$

Damit erhalten wir die Formel für die nächste Approximierte Nullstelle:

$$x^{(k+1)} := x^{(k)} - \frac{F\left(x^{(k)}\right)}{F'\left(x^{(k)}\right)}, \quad \text{falls } F'\left(x'(k)\right) \neq 0$$

Zurück zum Beispiel der Wurzel

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

Wir suchen Nullstellen von: $F(x) := x^2 - a$

Es gilt: $F'(x) = 2x$

Wir verwenden nun das Newton-Verfahren, wie wir es zuvor definiert haben:

$$x^{(k+1)} = x^{(k)} - \frac{F\left(x^{(k)}\right)}{F'\left(x^{(k)}\right)} = x^{(k)} - \frac{\left(x^{(k)}\right)^2 - a}{2x^{(k)}} = x^{(k)} - \frac{\left(x^{(k)}\right)^2}{2x^{(k)}} - \frac{a}{2x^{(k)}}$$

$$= x^{(k)} - \frac{x^{(k)}}{2} - \frac{a}{2x^{(k)}} = \frac{x^{(k)}}{2} - \frac{a}{2x^{(k)}} = \frac{1}{2}\left(x^{(k)} + \frac{a}{x^{(k)}}\right)$$

Die Fixpunktiteration von vorher ist ein Newton-Verfahren!

Problem:

Ableitung oft nicht bekannt

Sekantenverfahren

Sekantenverfahren: Idee

Gleich wie Newton-Verfahren, aber wir nähern die Ableitung in jedem Schritt neu an.

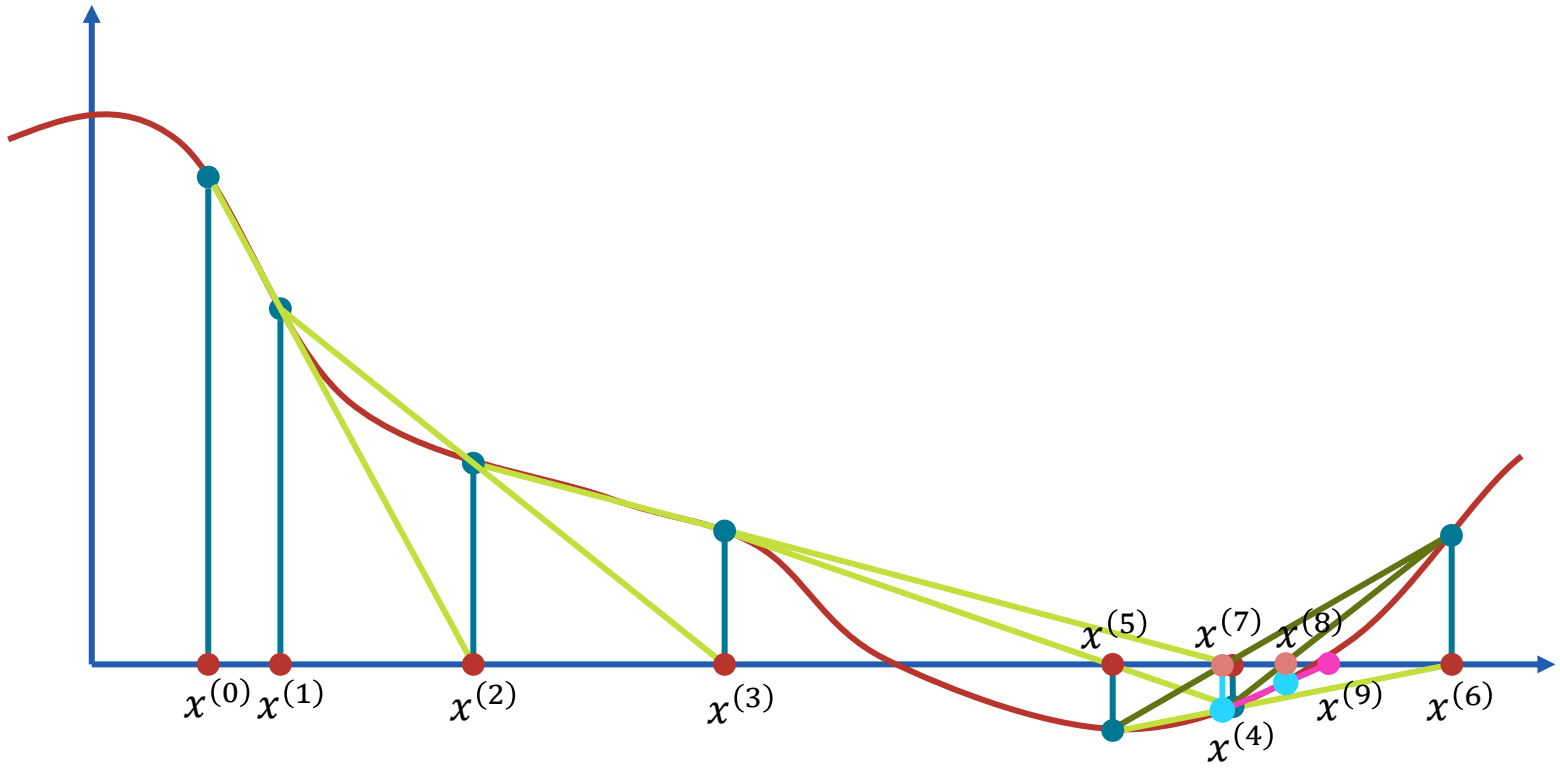
Wir definieren $q^{(k)} := \frac{F\left(x^{(k)}\right) - F\left(x^{(k-1)}\right)}{x^{(k)} - x^{(k-1)}}$ (Sekante zwischen den letzten beiden Punkten)

Nun verwenden wir dies als Annäherung für die Ableitung:

$$x^{(k+1)} := x^{(k)} - \frac{F\left(x^{(k)}\right)}{q^{(k)}}$$

(Verliert Sinn, wenn $q^{(k)} := 0$)

Sekantenverfahren: Visuell



Newton-Verfahren d-dimensional

Newton d-dimensional

Die ableitung in mehreren Dimensionen für $F: \mathbb{R}^d \to \mathbb{R}^d$ ist die **Jakobi-Matrix**:

$$D\mathbf{F}(\vec{x}) = \mathbf{J_F}(\vec{x}) = \left[\frac{\partial \mathbf{F}_j}{\partial \vec{x}_k}(\vec{x})\right]_{j=1,k=1}^{d,d}$$

Die partielle Ableitung jeder Komponente über jeden Parameter. Wie übersetzen wir nun die Newton-Formel?

$$x^{(k+1)} := x^{(k)} - \frac{F(x^{(k)}}{F'\left(x^{(k)}\right)} \quad \to \quad \vec{x}^{(k+1)} := \vec{x}^{(k)} - \mathbf{J_F}^{-1}\left(\vec{x}^{(k)}\right)\mathbf{F}\left(\vec{x}^{(k)}\right)$$

Wir setzen voraus, dass die Jakobi-Matrix invertierbar ist (Äquivalent von $dF(x) \neq 0$)

Lösen des Systems

Achtung: Anstatt $\mathbf{J_F}^{-1}\left(\vec{x}^{(k)}\right)\mathbf{F}\left(\vec{x}^{(k)}\right)$ direkt zu berechnen, schreiben wir:

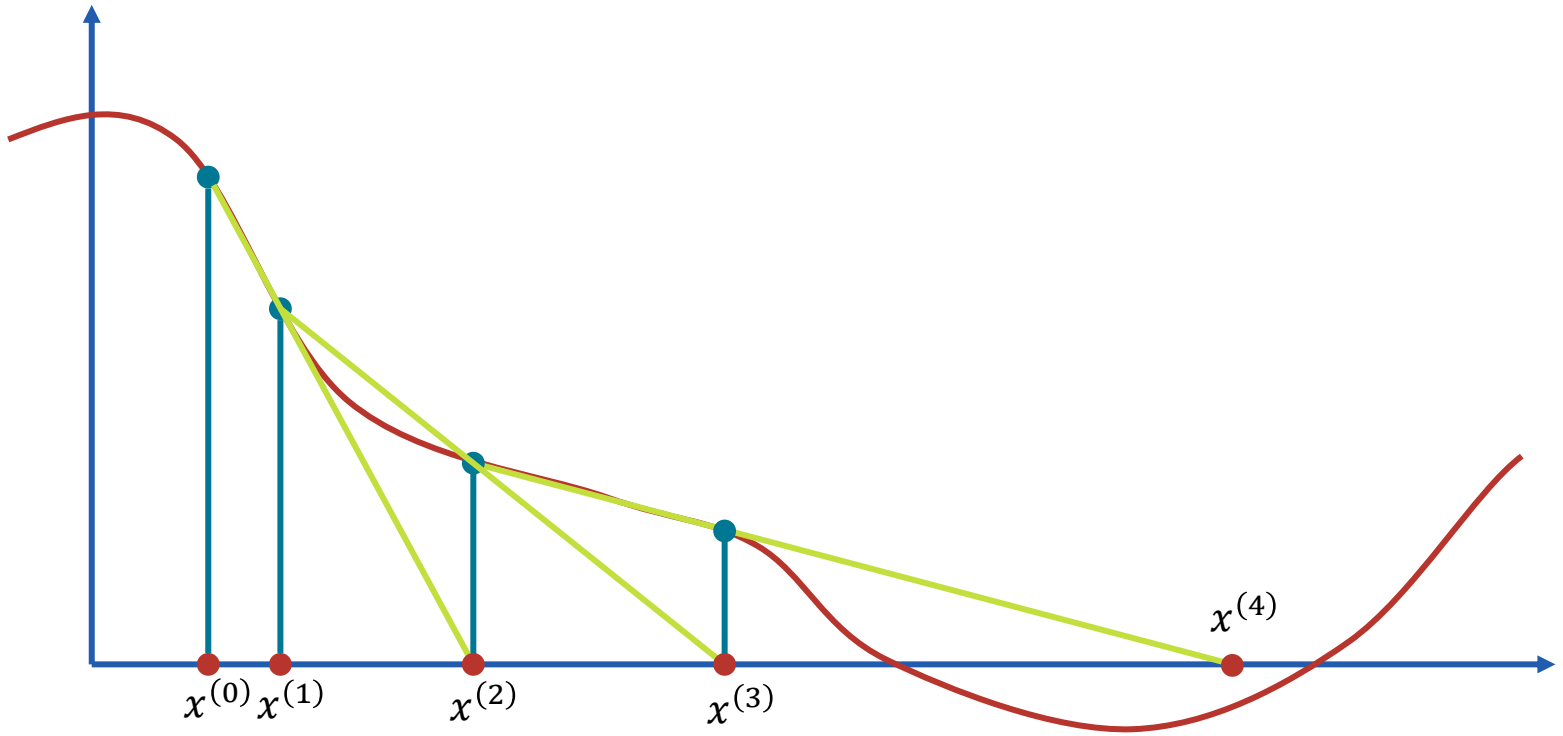
$$\mathbf{J_F}^{-1}\left(\vec{x}^{(k)}\right)\mathbf{F}\left(\vec{x}^{(k)}\right) = \mathbf{A} \quad \Longrightarrow \quad \mathbf{J_F}\left(\vec{x}^{(k)}\right)\mathbf{A} = \mathbf{F}\left(\vec{x}^{(k)}\right)$$

Und lösen das Gleichungssystem direkt ***oder*** mithilfe einer QR-Zerlegung nach $\mathbf{A}$ auf:

$$\mathbf{J_F}\left(\vec{x}^{(x)}\right) = \mathbf{QR} \quad \Longrightarrow \quad \mathbf{RA} = \mathbf{F}\left(\vec{x}^{(k)}\right)\mathbf{Q}^{-1}$$

Newton Gedämpft

Recall: Zu grosser Schritt



Gedämpfte Newton-Verfahren

Der letzte Schritt geht sehr weit! Wir wählen nun eine Lernrate δ. Nun ändern wir die Formeln:

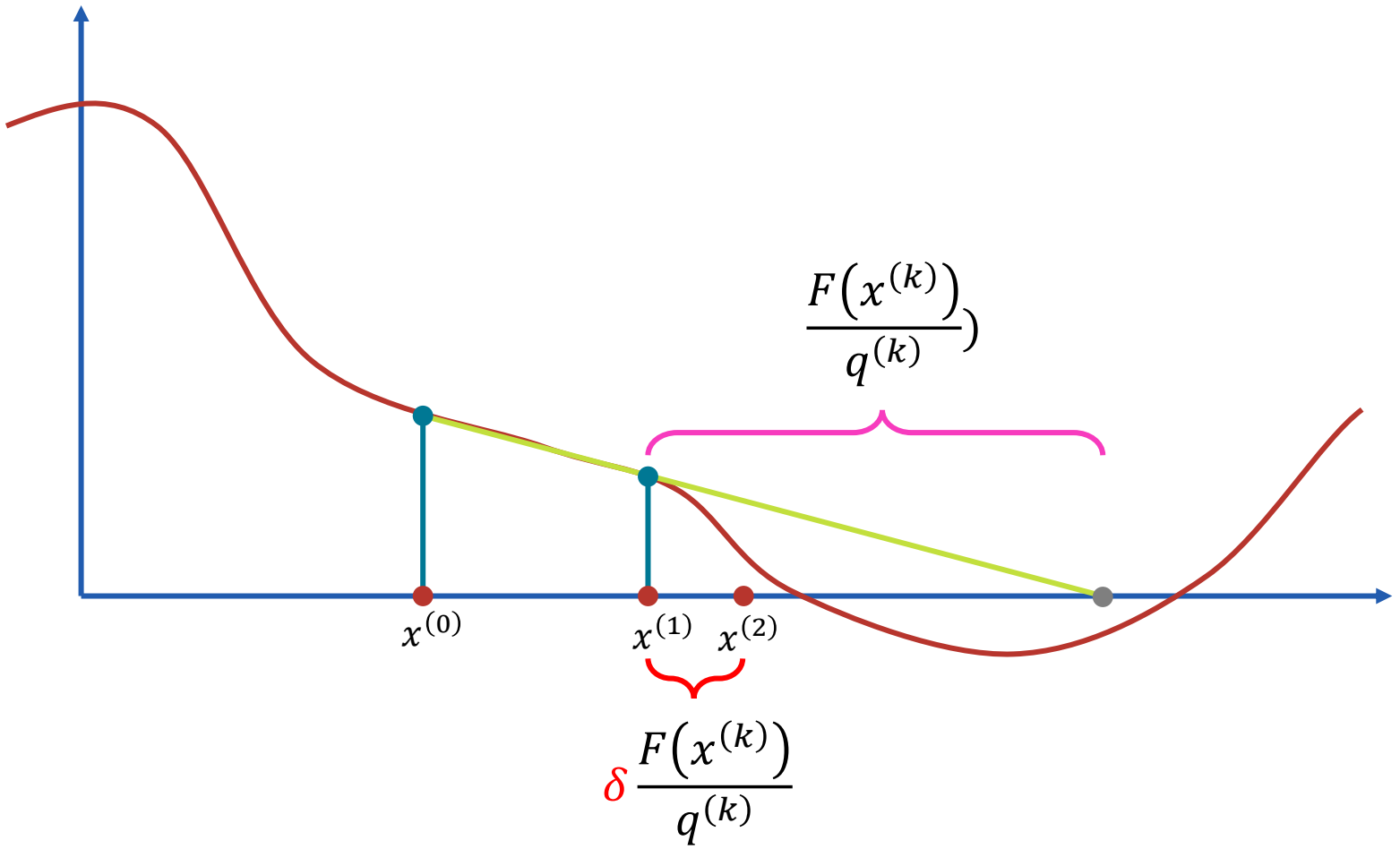
$$x^{(k+1)} := x^{(k)} - \delta \frac{F\left(x^{(k)}\right)}{F'\left(x^{(k)}\right)}$$

$$x^{(k+1)} := x^{(k)} - \delta \frac{F\left(x^{(k)}\right)}{q^{(k)}}$$

$$\vec{x}^{(k+1)} := \vec{x}^{(k)} - \delta \mathbf{J}_{\mathbf{F}}^{-1}\left(\vec{x}^{(k)}\right)\mathbf{F}\left(\vec{x}^{(k)}\right)$$

Wir gehen nur einen Teil des Schritts!

Sekantenverfahren: gedämpft



Quasi-Newton

Quasi-Newton: Idee

Was, wenn $\mathbf{J_F}(\mathrm{x})$ *zu teuer ist?*

Gibt es etwas ähnliches wie das Sekantenverfahren?

$$F'\left(x^{(k)}\right) \approx \frac{F\left(x^{(k)}\right) - F\left(x^{(k-1)}\right)}{x^{(k)} - x^{(k-1)}} = \mathbf{J}^{(k)}$$

Division durch Vektoren wird schwierig!

$$\mathbf{J_F}^{(k)} \cdot \left(x^{(k)} - x^{(k-1)}\right) = \mathbf{F}\left(x^{(k)}\right) - \mathbf{F}\left(x^{(k-1)}\right)$$

Können wir zu lösen versuchen, hat aber **mehrere Lösungen!**

Broyden Quasi-Newton

Wir definieren:

$$\Delta x^{(k)} := -\mathbf{J}_k^{-1}\mathbf{F}\left(x^{(k)}\right)$$

$$x^{(k+1)} := x^{(k)} + \Delta x^{(k)}$$

$$\mathbf{J}_{k+1} := \mathbf{J}_k + \frac{\mathbf{F}(x^{(k+1)}\left(\Delta x^{(k)}\right)^T}{\left\|\Delta x^{(k)}\right\|_2^2}$$

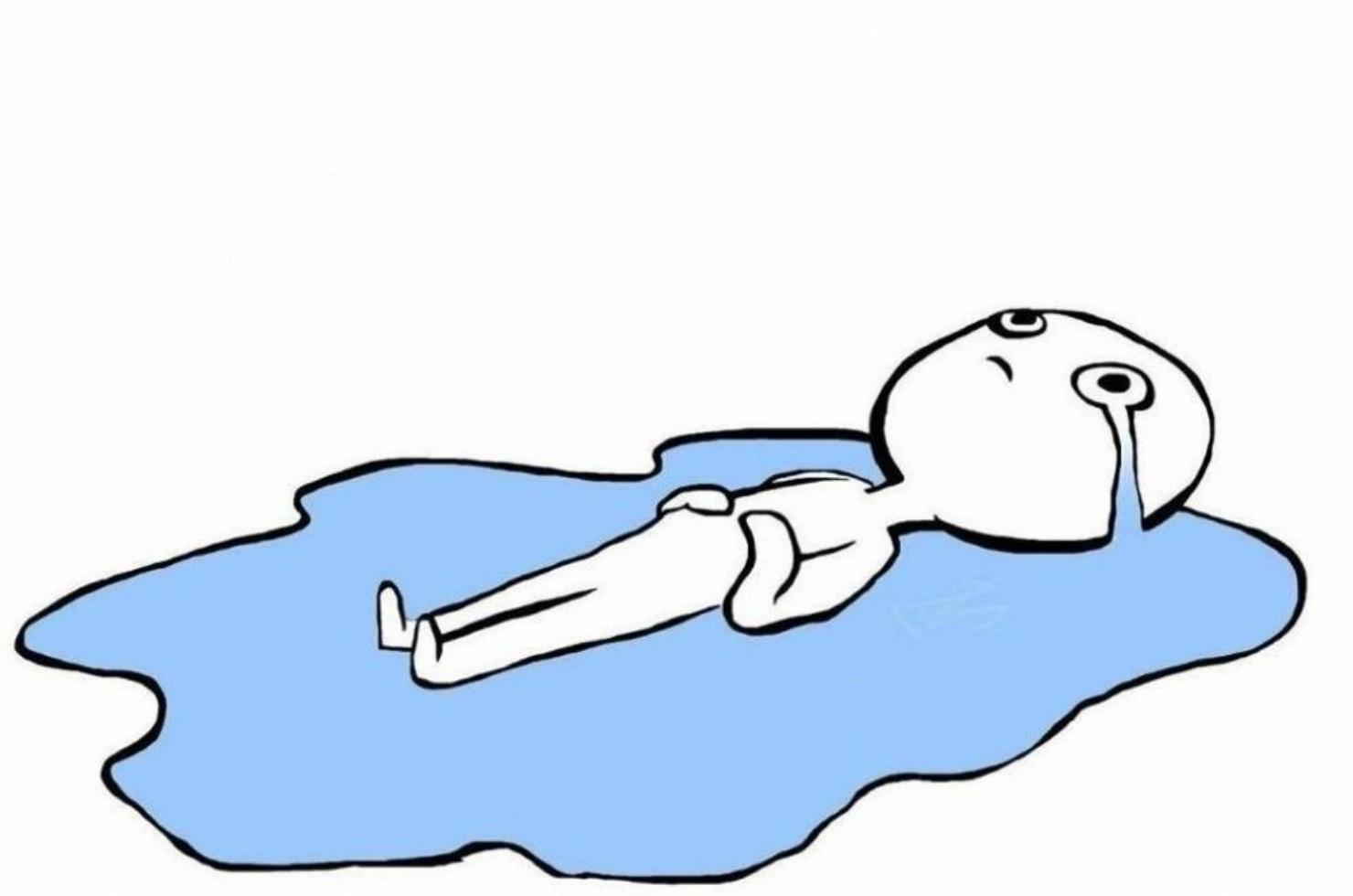
Wir definieren dabei $\mathbf{J}_0 = \mathbf{J}_\mathbf{F}\left(x^{(0)}\right)$

Luft anhalten...

Sherman-Morrison-Woodbury-Update-Formel

$$\mathbf{J}_{k+1}^{-1} = \left(\mathbf{I} - \frac{\mathbf{J}_k^{-1}\mathbf{F}\left(x^{(k+1)}\right)\left(\Delta x^{(k)}\right)^T}{\left\|\Delta x^{(k)}\right\|_2^2 + \underbrace{\left(\Delta x^{(k)}\right)^T \mathbf{J}_k^{-1}\mathbf{F}\left(x^{(k+1)}\right)}_{\text{Skalarprodukt}}} \right) \mathbf{J}_k^{-1}$$

…
AAAAHHHHH!!!



Skript to the rescue…

Code 6.9.4: Broyden-Verfahren

```
from scipy.linalg import lu_solve, lu_factor, norm, solve
from numpy import dot, zeros, zeros_like

def fastbroyd(x0, F, J, tol=1e-12, maxit=20):

    x = x0.copy() # make sure we do not change the iput
    lup = lu_factor(J) # LU decomposition of J

    k = 0; s = lu_solve(lup,F(x))  # start with a Newton corection
    sn = dot(s,s) # squared norm of the correction
    x -= s; f = F(x) # start with a full Newton step

    #containers for storing corrections s and their sn:
    dx = zeros((maxit,len(x))); dxn = zeros(maxit)
    dx[k] = s; dxn[k] = sn
    k += 1; # the number of the Broyden iteration

    #now we perform the Broyden iteration
    while sn > tol and k < maxit:
        w = lu_solve(lup,f)  #f = F(actual Broyden iteration x)
        #now we update the correction for the k-th step
        #using the Sherman-Morrison-Woodbury formel
        for r in range(1,k):
            w +=  dx[r]*( dot(dx[r-1],w) )/dxn[r-1]
        z = dot(s,w)
        s = (1+z/(sn-z))*w
        sn = dot(s,s)
        dx[k] = s; dxn[k] = sn
        x -= s; f = F(x); k+=1  #update x and iteration number k

    return x, k  #return the final value and the numbers of iterations needed
```

ETH zürich

Jonas Vonäsch
Teaching Assistant NumCS
jvonaesch@student.ethz.ch

jvonaesch.github.io